



UNIVERSITY OF ENGINEERING AND TECHNOLOGY MARDAN

NAME	RIFAQ AJMAL
REG NO	23MDBCS435
DEPPT	COMPUTER SCIENCE
SEMESTER	5th
SUBJECT	INFORMATION SECURITY
INSTRUCTOR	RAZA ULLAH

ASSIGNMENT NO 1

1. Intrusion Detection System (IDS)

An **IDS** is a security mechanism that monitors network traffic or system activities to detect suspicious behavior, unauthorized access, or attacks. AI-powered IDS systems are much more effective than traditional rule-based ones because they can **learn patterns** of normal vs. abnormal activity using **Machine Learning (ML)**.

2. Role of AI in IDS

- **Pattern Recognition:** AI learns what "normal" network traffic looks like and flags deviations.
- **Anomaly Detection:** Identifies unusual activities (e.g., port scanning, DDoS, SQL injection).
- **Classification:** Detects and categorizes attacks (e.g., DoS, Probe, R2L, U2R).
- **Adaptive Security:** Continues learning from new attack data, making the IDS smarter over time.

3. Machine Learning Algorithms for IDS

Some common ML algorithms used in IDS:

1. **Decision Trees (e.g., Random Forest, XGBoost)** ○ Good for classifying normal vs. attack traffic.
 - Can handle large datasets and noisy data.
2. **Support Vector Machines (SVM)**
 - Effective for binary classification (normal vs. intrusion).
 - Works well with high-dimensional feature sets (like network traffic logs).
3. **K-Nearest Neighbors (KNN)** ○ Simple and effective for small-scale IDS.
 - Classifies new traffic by comparing with past traffic patterns.
4. **Neural Networks (Deep Learning with TensorFlow/PyTorch)** ○ Learn complex attack patterns. ○ LSTMs or CNNs can analyze sequential packet data or log data for detecting advanced persistent threats (APTs).
5. **Clustering (Unsupervised Learning, e.g., K-Means, DBSCAN)** ○ Detect unknown/zero-day attacks without labeled data. ○ Groups suspicious activities into clusters.

4. Python Libraries for IDS

To build AI-based IDS in Python, you can use:

- **Scikit-learn** → For preprocessing, training ML models (Decision Trees, SVM, KNN).
- **TensorFlow / PyTorch** → For deep learning models like ANN, CNN, LSTM.
- **Pandas & NumPy** → For handling datasets (e.g., KDD Cup 99, NSL-KDD, CICIDS2017).
- **Matplotlib & Seaborn** → For visualization of results (ROC curve, confusion matrix).

- **Scapy** → For capturing and analyzing network packets in real-time.

5. Example Python Code (Simple IDS with Scikit-learn)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix

# Load dataset (e.g., NSL-KDD or CICIDS2017) data =
pd.read_csv("ids_dataset.csv")

# Features and labels
X = data.drop("label", axis=1)    # Features y =
data["label"]                    # Normal / Attack

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=42)

# Train Random Forest Classifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predictions
y_pred = model.predict(X_test)

# Evaluation
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

This example shows a **basic supervised IDS**. For advanced IDS, you can try **deep learning (TensorFlow)** or even **real-time monitoring with Scapy + ML models**.

Question 2

1. Malware analysis with Python

- Use Python libraries like pefile, lief, and androguard to extract features from malware samples.
- Explore Scikit-learn and TensorFlow to train and deploy classification models.

1. Malware Analysis Overview

Malware analysis is the process of examining suspicious files/programs to understand:

- **What it does** (steals data, encrypts files, opens backdoors, etc.).
- **How it behaves** (system calls, registry changes, network traffic). □ **How to detect it** using AI models.

AI + Python makes this process faster because instead of manually inspecting binaries, we can **automatically extract features** from malware and use ML/DL models to classify them as **benign vs. malicious**, or even into specific malware families (Trojan, Ransomware, Worm, etc.).

2. Python Libraries for Malware Feature Extraction

These libraries help analyze binary executables (like Windows PE files or Android APKs):

- **pefile** → ○ Extract metadata from Windows Portable Executables (PE).
 - Example: sections, imports, exports, DLLs, entry points.
- **lief** → ○ Analyze and modify PE, ELF, and Mach-O binaries. ○ Extract features like headers, instructions, and strings.
- **androguard** → ○ Focused on Android APKs, DEX, and ODEX files. ○ Extract permissions, API calls, and manifest features.
- **yara-python** (optional) → ○ Pattern matching library for malware signatures.

3. Extracted Features from Malware

Some examples of features used for training models:

- **Static Features (from binaries without running them):**
 - File size, number of sections, DLL imports, suspicious APIs (e.g., `CreateRemoteThread`, `VirtualAlloc`).
 - Entropy (measure of obfuscation or packing).
- **Dynamic Features (by executing malware in sandbox):**
 - System calls, registry changes, file I/O. ○ Network behavior (domains, IPs contacted).

4. Machine Learning for Malware Classification

After extracting features, AI models can classify malware:

- **Scikit-learn:**
 - Logistic Regression, Random Forest, SVM, Gradient Boosting.

- Best for structured tabular features.
- **TensorFlow / PyTorch:**
 - Deep Neural Networks (ANNs).
 - Convolutional Neural Networks (CNNs) on malware “images” (turning binaries into grayscale images).
 - Recurrent Neural Networks (RNNs/LSTMs) for analyzing sequential API calls.

5. Example Code: Static Malware Feature Extraction + Classification

```
import pefile
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Example: Extract features from a PE file
def extract_pe_features(filepath):
    pe = pefile.PE(filepath)
    features = {
        "num_sections": len(pe.sections),
        "entry_point": pe.OPTIONAL_HEADER.AddressOfEntryPoint,
        "image_base": pe.OPTIONAL_HEADER.ImageBase,
        "num_imports": len(pe.DIRECTORY_ENTRY_IMPORT) if hasattr(pe, "DIRECTORY_ENTRY_IMPORT") else 0
    }
    return features

# Load dataset of malware & benign files
files = ["malware1.exe", "benign1.exe"]
labels = [1, 0]
# 1 = malware, 0 = benign

data = [extract_pe_features(f) for f in files]
df = pd.DataFrame(data)
df["label"] = labels

# Split dataset
X = df.drop("label", axis=1)
y = df["label"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train classifier
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
```

6. Advanced Ideas

- **Deep Learning with TensorFlow:** Convert malware binaries into images and train a CNN.

- **Androguard Example:** Extract Android permissions (like `SEND_SMS`, `INTERNET`) and classify malicious APKs.
- **Hybrid Analysis:** Combine static + dynamic features for stronger detection.

In short:

- **pefile, lief, androguard** → feature extraction.
- **Scikit-learn, TensorFlow** → build and deploy malware classification models.

Question 03:

2. Network Security with Python

- Use AI to monitor and secure networks by analyzing network traffic
- Study Python libraries like Scapy, Nmap, and Zeek to capture and analyze network packets.
- Explore machine learning libraries like Scikit-learn and TensorFlow to build models for anomaly detection and vulnerability assessment.

1. Network Security with Python + AI

Network Security is about protecting systems and data as they move across networks. Python can play a **dual role** here:

1. **Traffic Capture & Analysis** (using libraries like **Scapy, Nmap, Zeek**).
2. **AI-driven Detection** (using **Scikit-learn, TensorFlow** to detect anomalies and vulnerabilities).

AI helps by spotting **anomalous traffic patterns**, detecting **intrusions**, and predicting **potential attacks** (e.g., DoS, MITM, port scans).

2. Python Libraries for Network Security

Scapy □ Packet crafting, sniffing, sending, and dissecting.

- Can capture live packets and analyze protocol headers (TCP, UDP, ICMP, HTTP, etc.).
- Example: Detect suspicious ICMP floods or unusual traffic.

Nmap (with python-nmap)

- Port scanning and vulnerability discovery.
- Identifies open ports, services, OS fingerprinting.
- Useful for vulnerability assessment.

Zeek (formerly Bro)

- A powerful **network analysis framework**.
- Parses network traffic into structured logs (HTTP, DNS, SSL, FTP, etc.).
- Python can process Zeek logs to detect anomalies.

3. Machine Learning in Network Security

After capturing traffic, AI models can be trained to detect:

- **Anomalies (unsupervised ML):** Detect traffic that deviates from normal patterns.
- **Intrusions (supervised ML):** Classify traffic as normal vs. attack (DoS, Probe, U2R, R2L).
- **Vulnerability Assessment:** Predict whether a given configuration/traffic pattern is exploitable.

Algorithms:

- **Unsupervised:** K-Means, DBSCAN, Isolation Forest (good for unknown threats).
- **Supervised:** Random Forest, SVM, Neural Networks.
- **Deep Learning:** LSTM (for sequential traffic), CNN (for feature maps), Autoencoders (for anomaly detection).

4. Example Workflows

(A) Packet Capture + Anomaly Detection with Scapy + Scikit-learn

```
from scapy.all import sniff, IP import
pandas as pd
from sklearn.ensemble import IsolationForest

packets_data = []

# Capture packets def
packet_callback(packet):
    if IP in packet:
        packets_data.append({
            "src": packet[IP].src,
            "dst": packet[IP].dst,
            "len": len(packet),
            "proto": packet[IP].proto
        })

sniff(prn=packet_callback, count=200) # Capture 200 packets
```

```
# Convert to DataFrame df =
pd.DataFrame(packets_data)

# Train anomaly detection model model =
IsolationForest(contamination=0.1)
model.fit(df[["len", "proto"]])

# Predict anomalies
df["anomaly"] = model.predict(df[["len", "proto"]]) print(df.head())
```

This script sniffs packets, extracts features, and detects anomalies in real-time.

(B) Port Scanning with Nmap

```
import nmap

nm = nmap.PortScanner() nm.scan('127.0.0.1',
'20-1000')
for host in nm.all_hosts():
    print("Host:", host, "State:", nm[host].state())
for proto in nm[host].all_protocols():
    print("Protocol:", proto)
    ports = nm[host][proto].keys()
    for port in ports:
        print("Port:", port, "State:",
nm[host][proto][port]['state'])
```

This helps in vulnerability assessment (finding open ports and services).

(C) Using Zeek Logs with ML

Zeek generates logs like `conn.log`, `dns.log`, etc. Python can read these logs and apply ML models.

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

# Example: Load Zeek connection log
df = pd.read_csv("conn.log", sep="\t", comment="#")

# Select features (duration, bytes, packets, etc.) X =
df[["duration", "orig_bytes", "resp_bytes"]].fillna(0) y =
df["label"] # Assume dataset is labeled

# Train model
clf = RandomForestClassifier() clf.fit(X,
y)
print("Model trained for intrusion detection!")
```

5. Advanced Ideas

- **Deep Learning IDS with TensorFlow:**

Use LSTMs to analyze sequential packet flows.

- **Real-time Threat Detection:**

Combine Scapy (live packet sniffing) + TensorFlow (prediction).

- **Hybrid IDS:**

Mix **signature-based** (known attacks) and **anomaly-based** (unknown threats).

In short:

- **Scapy** → Live packet capture + anomaly detection.
- **Nmap** → Port scanning + vulnerability discovery. □ **Zeek** → Log-based deep traffic analysis.

Scikit-learn / **TensorFlow** → AI for anomaly detection & vulnerability assessment.